

31-08-2026

Given an integer array `arr[]`, which denotes the positions of stalls. All the positions are distinct. There are `k` aggressive cows.

Assign the cows to the stalls such that the minimum distance between any two cows is maximized.

```
class Solution {
public:

    bool canCowsbePlacedWithGivenGap(vector<int> &arr, int k,int mid){
        int cows=1;
        int pos=arr[0];
        for(int i=1;i<arr.size();i++){
            if(arr[i]-pos>=mid){
                cows++;
                pos=arr[i];
            }
        }
        if(cows>=k){
            return true;
        }
        else{
            return false;
        }
    }

    int aggressiveCows(vector<int> &arr, int k) {
        int low=1;
        sort(arr.begin(),arr.end());//asc order
        int high=arr[arr.size()-1]-arr[0];
        int temp;
        while(low<=high){
            int mid=(low+(high-low)/2);

            if(canCowsbePlacedWithGivenGap(arr,k,mid)==true){
                temp=mid;
                low=mid+1;
            }
            else{
```

```

        high=mid-1;
    }
}
return temp;
}
};

```

Given an array of integers `nums` and an integer `threshold`, we will choose a positive integer divisor, divide all the array by it, and sum the division's result. Find the smallest divisor such that the result mentioned above is less than or equal to `threshold`.

Each result of the division is rounded to the nearest integer greater than or equal to that element. (For example: $7/3 = 3$ and $10/2 = 5$).

The test cases are generated so that there will be an answer.

```

class Solution {

public:

    bool isThisDivisorSatisfiesTheCondition(vector<int>& nums, int
threshold,int mid) {

        int sum=0;

        for(int i=0;i<nums.size();i++){

            sum+=(nums[i]/mid);

            if(nums[i]%mid!=0){

                sum+=1;

            }

        }

        return sum<=threshold;

    }

    int smallestDivisor(vector<int>& nums, int threshold) {

        int low=1;

```

```

int high=*max_element(nums.begin(),nums.end());

int temp=high;

while(low<=high){

    int mid=(low+(high-low)/2);

    if(isThisDivisorSatisfiesTheCondition(nums,threshold,mid)==true){

        temp=mid;

        high=mid-1;

    }

    else{

        low=mid+1;

    }

}

return temp;

}

};

```

Imagine you are standing inside a grand palace where 5 guards are on duty. Each guard is responsible for watching 5 large rooms. In every single one of these rooms, there are 5 adult tigers, and each adult tiger has 5 cubs.

Can you figure out how many legs are inside the palace?